

More Practical Non-interactive Encrypted Conjunctive Search with Leakage and Storage Suppression

Huu Ngoc Duc Nguyen¹[0009–0005–4022–7466] ^{*}, Shujie Cui¹[0000–0001–8124–6800],
Shangqi Lai²[0000–0002–0374–3593], Tsz Hon Yuen¹[0000–0002–0629–6792], and
Joseph K. Liu¹[0000–0001–6656–6240]

¹ Monash University, Clayton, VIC, Australia

² CSIRO Data61, Clayton, VIC, Australia

hngu0201@student.monash.edu, Shujie.Cui@monash.edu,
Shangqi.Lai@csiro.au, John.TszHonYuen@monash.edu,
Joseph.Liu@monash.edu

Abstract. Searchable symmetric encryption allows clients to outsource their databases to a semi-trusted cloud server while enabling private searches. The Oblivious Cross-Tag (OXT) protocol is a fundamental approach to conjunctive keyword search, ensuring that search performance scales with the least frequent keyword while introducing keyword pair result pattern (KPRP) and intersection result pattern (IP) leakages. However, recent studies show that the KPRP leakage in OXT can be exploited, allowing the cloud server to infer information about the client database. Several works have aimed to mitigate this issue, with Doris being the first non-interactive OXT-based scheme to hide KPRP and IP leakages. However, this comes at the cost of increased storage overhead. In this work, we propose a Doris-based conjunctive SSE scheme with improved storage efficiency. We replace the XOR filter in Doris with our XEBFF filter, which formalizes XOR filters and Binary Fuse Filters. Additionally, we introduce a frequency estimation approach using Count-Min Sketch to efficiently determine the least frequent keyword, which all previous OXT-based schemes overlook. Our scheme reduces storage overhead by 8% compared to Doris while maintaining search performance. With our s-term selection protocol, we ensure that search operations typically scale with the least frequent keyword.

Keywords: Symmetric Searchable Encryption · Conjunctive Search · Fingerprint-based Filter · Subset Predicate Encryption

1 Introduction

Searchable Symmetric Encryption (SSE) is a cryptographic solution that allows users to outsource encrypted datasets to a server and lets the server search for encrypted data without decrypting them. Early SSE research primarily focused on

^{*} Corresponding author

single keyword searches, which is not expressive for many practical applications. Several works study conjunctive SSE [1, 3, 7, 17, 5]. Among them, the “Oblivious Cross-Tags” (OXT) scheme stands out as a foundational work, influencing subsequent research on this topic.

Overview of OXT. The OXT scheme is the first SSE scheme that achieves sublinear search performance for conjunctive queries. It works by dividing a conjunctive query $\Phi = (w_1, w_2, \dots, w_q)$ into *s-term*, the estimated least frequent keyword in the database, and *x-term*, the remaining keywords. The encrypted database of OXT consists of a TSet, an encrypted inverted index, and a XSet, an array of all encrypted (w, id) (i.e., keyword and document identifier) pairs in the database. Upon receiving the query, the server first retrieves document identifiers containing the s-term from TSet and then verifies whether these identifiers contain x-terms by checking if each pair of them is a member of XSet in a secure manner. Therefore, the computational cost scales with the number of document identifiers associated with the s-term, rather than the total number of documents in the database, thus achieving sublinear search performance.

Selection of S-terms. Existing OXT-based conjunctive SSE schemes mainly focus on reducing leakage and improving performance. However, they overlooked an important issue: how to efficiently find the s-term for each query. As mentioned, the performance of OXT-based approaches heavily depends on the s-term, as it determines the efficiency of the membership checking. Cash et al. [5] provide a limited discussion on choosing an optimal s-term, e.g., one can use general attribute statistics of relational databases, such as preferring last names over genders in the queries as the s-term, or group keywords by frequency via Bloom filters, and let client use this frequency to select the s-term. However, those solutions are only applicable on specific scenarios (e.g., relational databases).

Effectively Suppress Leakage of OXT. Another issue of OXT is leakage. One of the most notable leakage is the keyword pair result patterns (KPRP) leakage. Particularly, let $DB(w_i)$ represents the documents containing the keyword w_i . For a query $\Phi = (w_1, w_2, \dots, w_q)$ where w_1 is the s-term, KPRP means adversaries can learn $DB(w_1) \cap DB(w_i)$ for $i \in [2, q]$, in addition to the search result. Zhang et al. [19] demonstrate that KPRP severely compromises OXT scheme, enabling full recovery of client query keywords with 100% accuracy.

To conceal KPRP, Lai et al. [15] propose an enhanced version of OXT called Hidden Cross Tags (HXT) which uses Bloom Filter to build XSet and encrypts it with Symmetric Hidden Vector Encryption (SHVE), a form of predicate encryption. However, HXT requires an additional communication round compared to OXT to send a subset of the Bloom Filter to the client who then computes HVE tokens to complete the search. Moreover, the Bloom Filter requires 44% additional storage to achieve an optimal false positive rate [2]. To reduce the storage cost, Ma et al. [16] propose another OXT-based scheme called Practical Hidden-Cross Tag (PHXT) by introducing a new cryptographic primitive called Hash-based Subset Membership Checking. While the scheme reduces storage size by 91.29% compared to HXT, it still requires extra communication rounds.

Table 1: Comparison with Existing Conjunctive SSE Schemes.

Schemes	Storage	EDB Setup	Search	Rounds	KPRP hidden	IP hidden	Term selections
OXT [5]	$O(n)$	$O(T_{gen}) + O(n)\mathcal{E}$	$O(T_q) + O((q-1)n_{w_1})\mathcal{E}$	1	No	No	No
HXT [15]	$O(n + \nu pn)$	$O(T_{gen}) + O(n)(\mathcal{E} + \mathcal{P} + H)$	$O(T_q) + O((q-1)n_{w_1})(\mathcal{E} + H)$	2	Yes	No	No
Doris [18]	$O(n + \gamma N)$	$O(T_{gen}) + O(N)(\mathcal{P} + H + XF)$	$O(T_q) + O((q-1)n_{w_1})(\mathcal{P} + \mathcal{X})$	1	Yes	Yes	No
Ours	$O(n + \omega N)$	$O(T_{gen}) + O(N)(\mathcal{P} + H + XEBFF)$	$O(T_q) + O((q-1)n_{w_1})(\mathcal{P} + \mathcal{X})$	1	Yes	Yes	Yes

Let $\Phi = w_1 \wedge w_2 \wedge \dots \wedge w_q$ be the query. $n = \sum_{w \in W} |DB(w)|$ is the number of document-keyword pairs. $N = \sum_{1 \leq i \leq j \leq |W|} |DB(w_i) \cap DB(w_j)|$ is the number of unique keyword pairs. $n_{w_1} = |DB(w_1)|$ is the number of document matching s-term. p and ν are the false positive rate and parameter of Bloom Filter, respectively. γ and ω are configurable parameters of XOR filter and XEBFF, respectively, where $\omega \leq \gamma$. $\mathcal{E}$ is the modular exponentiation, $\mathcal{P}$ is the PRF operation. H is the hash function operation. $\mathcal{X}$ is the exclusive-or operation, T_{gen} and T_q are the computational cost of $TSet$ generation and search, respectively. XF and $XEBFF$ are setup operations of XOR Filter and XOR-Extended Binary Fuse Filter, respectively.

Lately, Wang et al. [18] introduce Doris, the first non-interactive KPRP-hiding conjunctive SSE scheme. Doris replaces SHVE with Symmetric Subset Predicate Encryption (SSPE) employing XOR filter, a fingerprint-based probabilistic data structure, for membership checking. With the novel design, Doris removes the expensive exponentiation computation for membership checking. Moreover, it avoids another type of leakage called keyword cross-query intersection result patterns (IP), where the adversaries can learn $DB(w_1) \cap DB(w'_1)$ of queries with different s-terms but the same x-terms, e.g., $\Phi = (w_1, w_2, \dots, w_q)$ and $\Phi' = (w'_1, w_2, \dots, w_q)$, where $w_1 \neq w'_1$. Moreover, Doris incurs lower storage overhead, with only 23% extra storage compared to HXT's 44%.

Our Contributions. In this work, we propose a new OXT-based non-interactive conjunctive SSE that can also hide both KPRP and IP. Compared with Doris, we further reduce the storage overhead by 50% and decrease the encryption time of SSPE by 1.5x. This is achieved by replacing the XOR filter with a more efficient fingerprint-based probabilistic data structure called XOR-Extended Binary Fuse Filter (XEBFF). Doris's XOR filter bounds the storage cost to $1.23n$, where n is the number of elements in the XSet. Our new data structure XEBFF, which integrates the constructions of XOR filters and Binary Fuse Filters (BFF) achieves a lower storage overhead of $1.125n$ for large datasets. BFF, an alternative design of the XOR filter, reduces storage costs by partitioning the array into hundreds of small, same-size non-overlapping segments, unlike XOR filters, which divide the array into only three segments. However, BFF is impractical for small datasets due to its segmentation strategy. To address this, XEBFF dynamically adjusts segmentation based on dataset size: for XSet with more than 10^7 elements, it partitions into hundreds of segments, while for smaller XSet, it defaults to three segments. Moreover, this generalized fingerprint-based probabilistic data structure can be seamlessly integrated with SSPE without increasing false negative rate, enhancing the efficiency and scalability of our solution. A detailed comparison of our scheme with existing works is presented in Table 1.

More importantly, we introduce a formal protocol for acquiring s-terms using the Count-Min Sketch (CMS) data structure, a space-efficient probabilistic structure that provides quick access to approximate frequency counts. We construct a frequency table using CMS, allowing the client to locally determine s-terms without interacting with the server for each query. By integrating CMS into our protocol, we enable rapid s-term selection with minimal computational cost. Its lightweight design ensures that the client can efficiently maintain the frequency table on the client side, reducing storage and processing overhead while improving query efficiency. Our contributions are summarized as follows:

- To reduce storage overhead of XOR filter, we introduce the extended XOR-Binary Fuse Filter (XEBFF). This extended filter can be effortlessly integrated with the generic construction of Doris. Consequently, our proposed non-interactive scheme achieves the same security as Doris by concealing both IP and KPRP leakages while reducing additional storage overhead.
- We design a frequency table which enables the client to query the approximate frequency of keywords in the database while requiring minimal storage overhead. This allows the client to efficiently determine the least frequent keyword. To the best of our knowledge, this is the first OXT-based SSE scheme that employs a s-term selection protocol.
- We implement a prototype of our scheme and evaluate its performance. Our SSPE achieves about $1.5\times$ faster encryption and roughly 8% lower storage costs than Doris on large datasets.

2 Related Work

A naïve approach to conjunctive search is to perform a single keyword search for each keyword and then intersect the results [5]. However, this method presents significant drawbacks. Firstly, search performance is proportional to the number of documents matching each keyword. Secondly, the approach reveals keyword-specific results and their intersections, potentially allowing the server to infer document contents through statistical analysis. These limitations have motivated extensive research into more efficient and secure conjunctive search methods.

Golle et al. [7] propose two pioneering conjunctive search schemes. The core idea is to define specific keyword fields for all documents. The first scheme ensures a constant online communication cost; however, it requires the trapdoor size to grow linear with the total number of documents. Although part of the trapdoor can be precomputed, the size is undesirable. The second scheme guarantees constant communication cost but requires the trapdoor to be linear in keyword fields. Both schemes rely on expensive modular exponentiations for encryption, trapdoor generation, and search. Additionally, the communication and storage costs for a query grow linearly with the number of documents in the database, making these schemes inefficient. Based on the concept of keyword fields, Ballard et al. [1] propose two conjunctive search schemes, using Shamir’s Secret Sharing and bilinear pairing, respectively. These schemes require less storage than earlier proposals, but the size of the trapdoor remains proportional to

Table 2: Notations and Terminologies

Notation	Meaning
λ	Security Parameter
U	Universe of all possible elements
$S, S $	Set of elements from the universe U , $S \subseteq U$, and its cardinality
B	Fingerprint array, and its size $ B = b$
h_f	Random hash function mapping elements of U to k -bit values
m	Number of segments
s	The s -th segment of array B ; $s \in \mathbb{N}$ and $0 \leq s < m$
$\mathcal{H}$	A collection of hash functions of XEBFF
id_i	Document identifier of i -th document
d	Number of documents in the database
W_i	Set of keywords in document id_i
W	Set of all keywords in the database
DB	Database, $DB = \{(id_i, W_i)_{i=1}^d\}$
$DB(w_i)$	Set of documents containing w_i
$\Phi(\bar{w})$	Conjunctive query
$\mathcal{Q}$	Set of conjunctive query
msk	Master key
α	Number of hash functions in Count-Min Sketch
β	Range of value of the hash functions in Count-Min Sketch

the number of documents being searched. Wang et al. [17] introduce a scheme that supports keyword-field-free conjunctive search using bilinear map for every keyword and document index. Therefore, the trapdoor can be constructed with any combination of keywords, regardless of their position. However, it results in the trapdoor size being linear in the number of keywords in the database.

OXT is the first scheme that achieves sublinear search complexity for conjunctive queries. However, this improvement comes with leakages. Zhang et al. [19] have demonstrated OXT's susceptibility to file injection attacks. In this attack, Zhang exploited KPRP to learn client query keywords without knowledge of the documents on the server with 100% accuracy. Motivated by this problem, subsequent research is extensively studied to improve the performance and security of OXT, especially in addressing file injection attacks. One notable improvement called HXT is proposed by Lai et al. [15]. HXT utilizes Bloom Filters and SHVE to eliminate KPRP leakage. However, Ma et al. [16] identify significant storage and communication overhead in this approach due to SHVE. To address this, they proposed the PHXT protocol, which replaces SHVE with a more efficient hash-based subset membership checking technique for XSet encryption and querying. As a result, PHXT significantly reduces the storage size and the communication overhead by 91.29%, 64.29% compared to HXT. More recently, Wang et al. [18] introduced Doris, the first conjunctive encrypted multi-map protocol, leveraging SSPE to address both KPRP and IP leakage. The scheme introduces a novel approach to building XSet, making it a non-interactive scheme.

3 Preliminaries

A list of notations and terminologies are summarized in Table 2. Following this, we introduce essential cryptographic primitives to construct our scheme.

3.1 Symmetric Subset Predicate Encryption

Subset Predicate Encryption is introduced by Katz et al. [13], allowing to check whether a set X is a subset of Y within public-key settings. It has been later extended to the symmetric settings as SSPE by Wang et al. [18]. SSPE consists of four PPT algorithms. $Setup(1^\lambda)$ generates master secret msk , the message space $\mathcal{M}$ and a universe $\mathcal{U}$, $KeyGen(msk, X)$ derive a decryption key sk_X for a predicate set $X \subseteq \mathcal{U}$, $Enc(msk, msg, Y)$ encrypts the message msg under the attribute set $Y \subseteq \mathcal{U}$, producing ciphertext ct . $Dec(sk_X, ct)$ decrypts ct with sk_X , producing msg if $X \subseteq Y$ or $\perp$ otherwise.

Correctness. The encrypted message can be correctly decrypted if and only if $X \subseteq Y$. Specifically, if $X \subseteq Y$, $Pr[Dec(sk_X, ct = msg)] = 1$, otherwise if $X \not\subseteq Y$, $Pr[Dec(sk_X, ct = \perp)] = 1 - negl(\lambda)$.

Security. Let $\mathcal{L}$ be the leakage profile of SSPE, an SSPE is considered $\mathcal{L}$ -selective secure if for all PPT adversaries $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ such that $Pr[\mathbf{Real}_A^{SSPE}(\lambda) = 1] - Pr[\mathbf{Ideal}_{A,\mathcal{S}}^{SSPE}(\lambda) = 1] \leq negl(\lambda)$ where $\mathbf{Real}_A^{SSPE}(\lambda)$ and $\mathbf{Ideal}_{A,\mathcal{S}}^{SSPE}(\lambda)$ are defined as follows.

$\mathbf{Real}_A^{SSPE}(\lambda)$:

- **Setup phase:** The adversary $\mathcal{A}$ chooses attribute set Y and sends to challenger. Challenger runs $SSPE.Setup(1^\lambda)$ and gives message space $\mathcal{M}$ to $\mathcal{A}$.
- **Query phase 1:** The adversary $\mathcal{A}$ obtains private keys sk_{X_i} for an arbitrary set X_i where $i \in 1, \dots, q$.
- **Challenge phase:** The adversary $\mathcal{A}$ outputs the message $msg \in \mathcal{M}$ and obtains the ciphertext $ct \leftarrow SSPE.Enc(msk, msg, Y)$.
- **Query phase 2:** The adversary may continue to request private keys for arbitrary sets. Finally, the adversary outputs a guess.

$\mathbf{Ideal}_{A,\mathcal{S}}^{SSPE}(\lambda)$:

- **Setup phase:** The adversary $\mathcal{A}$ chooses the set of attributes Y . Simulator $\mathcal{S}$ runs $Sim.Setup()$ and gives message space $\mathcal{M}$ to $\mathcal{A}$.
- **Query phase 1:** The adversary $\mathcal{A}$ obtains private keys sk_{X_i} for an arbitrary set X_i where $i \in 1, \dots, q$, $sk_{X_i} \leftarrow Sim.KeyGen()$.
- **Challenge phase:** The adversary $\mathcal{A}$ outputs the message $msg \in \mathcal{M}$ and obtains the ciphertext $ct \leftarrow Sim.Enc()$.
- **Query phase 2:** The adversary may continue to request private keys for arbitrary sets. Finally, the adversary outputs a guess.

3.2 Searchable Symmetric Encryption

In a single-writer single-reader setting, the data owner (client) encrypts the plaintext database and sends it to the cloud server (server), which stores the encrypted database. When the client searches for keywords, search tokens are generated based on desired keywords. The server uses the tokens to retrieve the matching encrypted documents without decrypting either tokens or documents. The encrypted documents are sent back to the client, who decrypts them using

private key, ensuring that the server learns nothing about the queries or the results. Let λ be the security parameter, SSE scheme Π consists of two algorithms. **EDBSetup** $(1^\lambda, DB) \rightarrow (param, K, EDB)$: The algorithm generates an encrypted database EDB , public parameters $param$ and a set of secret keys K , where EDB is then sent and stored on the server while K is kept by the client. **Search** $(K, \Phi(\bar{w}), EDB) \rightarrow (DB(\Phi(\bar{w})))$: It is a protocol run between client and server. Client's input are set of secret keys K and query $\Phi(\bar{w})$, while the server's input is EDB . At the end of the protocol, the client outputs the set of identifiers $DB(\Phi(\bar{w}))$ matching query $\Phi(\bar{w})$, while the server outputs nothing.

Correctness. The scheme Π is computationally correct if for all PPT adversaries $\mathcal{A}$ can win the game $\mathbf{Cor}_\mathcal{A}^\Pi$ with negligible probability $\Pr[\mathbf{Cor}_\mathcal{A}^\Pi = 1] \leq \text{negl}(\lambda)$. In this game, adversary $\mathcal{A}$ selects its own plaintext database DB and obtains encrypted EDB using $EDBSetup$. Then the adversary adaptively chooses conjunctive queries $\Phi(\bar{w})$ and runs $Search(\Phi(\bar{w}), EDB)$ to get the result Res . If Res and $DB(\Phi(\bar{w}))$ are different, the game outputs 1. Otherwise, it outputs 0.

Security. As defined in [5], The security definition guarantees that no information about queries and database is revealed beyond the predefined leakage profile. Let $\mathcal{L}$ be the leakage profile of SSE Π , an SSE Π is considered $\mathcal{L}$ -semantically secure against (non)-adaptive attacks if for all PPT adversaries $\mathcal{A}$ there exists a PPT simulator S such that $\Pr[\mathbf{Real}_\mathcal{A}^\Pi(\lambda) = 1] - \Pr[\mathbf{Ideal}_{\mathcal{A},S}^\Pi(\lambda) = 1] \leq \text{negl}(\lambda)$ where $\mathbf{Real}_\mathcal{A}^\Pi(\lambda)$ and $\mathbf{Ideal}_{\mathcal{A},S}^\Pi(\lambda)$ are defined as follows. **Real** $_\mathcal{A}^\Pi(\lambda)$: The adversary $\mathcal{A}(1^\lambda)$ chooses a database DB . The experiment runs $EDBSetup(1^\lambda, DB)$ and gives EDB to $\mathcal{A}$. $\mathcal{A}$ (non)-adaptively selects a query $\Phi[i] \in \mathbf{Q}$, the experiment runs the $Search(mk, \Phi[i], EDB)$ and stores the transcript and client's output in $t[i]$. Finally, EDB and $t[i]$ are given to $\mathcal{A}$, which returns a bit that the game uses as its own output. **Ideal** $_{\mathcal{A},S}^\Pi(\lambda)$: The adversary $\mathcal{A}(1^\lambda)$ chooses a database DB . The experiment runs $S(\mathcal{L}(DB))$ and gives EDB to $\mathcal{A}$. $\mathcal{A}$ (non)-adaptively selects a query $\Phi[i] \in \mathbf{Q}$, the experiment runs $S(\mathcal{L}(DB, \mathbf{Q}))$ and gives its outputs to $\mathcal{A}$, which returns a bit that the game uses as its own output.

3.3 Count-Min Sketch

Algorithm 1 Count-Min Sketch

<u>CMS.Setup</u>(α, β) 1: Create a 2D array C of size $\alpha \times \beta$, initialized to 0 2: Generate α hash functions $\mathcal{H} = \{H_1, H_2, \dots, H_\alpha\}$ 3: return $(C, \mathcal{H})$ <u>CMS.Add</u>(x) 1: for $i = 1$ to α do 2: $j \leftarrow H_i(x) \bmod \beta$	3: $C[i][j] \leftarrow C[i][j] + 1$ 4: end for 5: return C <u>CMS.Query</u>(x) 1: Initialize $minCount$ to ∞. 2: for $i = 1, \dots, \alpha$ do 3: Calculate index $j \leftarrow H_i(x) \bmod \beta$. 4: $minCount \leftarrow \min(minCount, C[i][j])$. 5: end for 6: Return $minCount$.
---	---

CMS [6] is a data structure that efficiently estimates the frequency of elements in a stream. Our work uses CMS to maintain a frequency table with fixed-size storage cost, making it suitable for client with limited storage. In detail, as shown in Algorithm 1, $CMS.Setup(\alpha, \beta)$ initializes a 2D array C of size $\alpha \times \beta$ where α is the number of hash functions and β denotes the range of possible outputs of the hash functions. $CMS.Add(x)$ updates the frequency table for an element x by hashing it with each hash function H_i to calculate the index j in the corresponding row and increment the value at $C[i][j]$. $CMS.Query(x)$ estimates the frequency of an element by hashing x with each hash function, retrieving the corresponding values in the table, and returning the minimum value across all rows. The accuracy of CMS depends on α and β . Let e be the base of the natural logarithm function, the width β determines the additive error bound $\epsilon = e/\beta$, with larger values reducing the estimation error. The depth α controls the confidence level, where the probability of exceeding the error bound is at most $\delta = e^{-\alpha}$. Selecting suitable values for α and β allows flexible trade-off between accuracy, and storage cost, making it particularly appropriate for client-side applications with limited memory and tolerance for approximate results.

4 XOR-Extended Binary Fuse Filter

Algorithm 2 XEBFF

<pre> XEBFF.setup(b, m) 1: $B \leftarrow \emptyset$, where $B = b$ 2: $h_f : U \rightarrow \{0, 1\}^k$ 3: $\mathcal{H} = \{h_0, h_1, \dots, h_{\kappa} - 1\}$ 4: return $B, h_f, \mathcal{H}$ XEBFF.map($S, \mathcal{H}$) 1: Initialize empty stacks P and Q 2: Sort keys $x \in S$ based on value of $h_0(x)$ 3: Initialize empty array T, $T = b$ 4: for $x \in$ sorted set S do 5: for $h \in \mathcal{H}$ do 6: $T[h(x)] \leftarrow T[h(x)] \cup x$ 7: end for 8: end for 9: for $h \in \mathcal{H}$ do 10: if $T[h] = 1$ then 11: $Q \leftarrow i$ 12: end if 13: end for 14: while $Q \neq \text{null}$ do 15: $i \leftarrow Q$ 16: $x \leftarrow T[i]$ 17: $P \leftarrow (x, i)$ 18: for $h \in \mathcal{H}$ do 19: $T[h(x)] \leftarrow T[h(x)] \setminus \{x\}$ 20: if $T[h(x)] = 1$ then 21: $Q \leftarrow h(x)$ </pre>	<pre> 22: end if 23: end for 24: end while 25: if $P \neq S$ then 26: XEBFF.setup() 27: XEBFF.map() 28: end if 29: return P XEBFF.construct($B, h_f, S, \mathcal{H}$) 1: $P \leftarrow XEBFF.map()$ 2: for $(x, i) \in P$ do 3: $B[i] \leftarrow h_f(x)$ 4: for $h \in \mathcal{H}$ do 5: if $h(x) \neq i$ then 6: if $B[h(x)] = \text{null}$ then 7: $B[h(x)] \xleftarrow{\\$} \{0, 1\}^k$ 8: end if 9: $B[i] \leftarrow B[i] \oplus B[h(x)]$ 10: end if 11: end for 12: end for 13: for $j \in B$ do 14: if $B[j] = \text{null}$ then 15: $B[j] \xleftarrow{\\$} \{0, 1\}^k$ 16: end if 17: end for 18: return B </pre>
---	---

Fingerprint-based filters are used for membership testing. For each key in a set, a k -bit fingerprint is generated using a hash function $h(x) \rightarrow \{0, 1\}^k$ and stored in a fingerprint table. To test the membership of a candidate key y , we compute its fingerprint $h(y)$ and compare it against the entries in the table. A match indicates that y is in the set, while no match implies it is not. XOR Filter [9] and Binary Fuse Filter (BFF) [11] are efficient fingerprint-based structures with a false positive rate of $1/2^k$. While both filters share similar concept and construction method, BFF sorts the array prior to construction, whereas the XOR Filter uses random access. This distinction results in the XOR filter having higher construction times. Additionally, to optimize memory locality, BFF divides the array into multiple segments (i.e. hundreds), while the XOR filter partitions the array into exactly three segments. As a result, XOR Filter has a storage cost of $1.23n$, while BFF lowers it to $1.125n$ where n is the number of elements in the dataset. However, the size of each segment in BFF must be sufficiently large, approximately $2^{\log_{3.33} n + 2.25} \approx 4.8n^{0.58}$ [11], to ensure a high construction success rate. As a result, BFF is most effective for large datasets (i.e., those exceeding 10^7 elements). For smaller datasets, XOR Filter is preferable. We introduce the XEBFF, which integrates and formalizes the construction principles of XOR Filters and BFF for any dataset size. Given a universe U , XEBFF builds a fingerprint array B for efficient membership checking. It consists of the following algorithms, the details of which are shown in Algorithm 2.

XEBFF.setup(b, m): Initializes an empty array B of size b , and partitioned into m equal segments. It also defines κ hash functions $h_0, h_1, \dots, h_{\kappa-1}$, mapping values in U to κ consecutive segments. The expression is as follows:

$$\mathcal{H} = \{h_t : U \rightarrow \left[t \cdot (s+1) \cdot \frac{b}{m}, (t+1) \cdot (s+1) \cdot \frac{b}{m} \right) \mid s \in \mathbb{N}, 0 \leq s < m, t \in [0, \kappa-1]\}$$

XEBFF.map($S, \mathcal{H}$): This algorithm maps elements of S using collection of hash functions $\mathcal{H}$. It initializes an empty stack P , sorts S by $h_0(x)$, and creates an empty array T of size $|S|$. Each element in S is added into T whose indices are the hash values of κ hash functions as shown on line 4 of Algorithm 2. Due to hash collision, some locations in T contain more than one element. For locations in T that contain exactly one element (primary location), these locations and their corresponding values are added to the stack P . The associated values are then removed from all other locations in T . This process repeats until all elements are assigned. If P does not match $|S|$, the algorithm restarts with a new collection of hash functions from $XEBFF.setup()$.

XEBFF.construct($B, h_f, S, \mathcal{H}$): The algorithm runs $P \leftarrow XEBFF.map(S, \mathcal{H})$, for each $(x, i) \in P$, and it updates $B[i] \leftarrow \bigoplus_{t \in [0, \kappa-1] \setminus \{t'\}} B[h_t(x)] \oplus h_f(x)$ where $h_{t'}(x) = i$ and $h_f : U \rightarrow \{0, 1\}^k$.

Construction Success Rate. According to [9, 11], there is a probability that the filter construction may fail due to the hash collision. However, experimental results demonstrate that the filter can be successfully constructed with a 100%

success rate when $|S| \geq 10^7$. For smaller datasets ($|S| < 10^7$), the success rate exceeds 80%, when the size of the array B is set to $1.23 \cdot |S| + 32$.

Parameter choices. To achieve the optimal storage cost, empirical evidence from [9] [11] suggests that the optimal number of hash functions is three. The size of the array B and number of segments m are determined as follows:

$$b \approx \begin{cases} 1.23 \cdot |S| + 32 & \text{if } |S| < 10^7 \\ 1.125 \cdot |S| & \text{otherwise.} \end{cases}, m = \begin{cases} 3 & \text{if } |S| < 10^7 \\ 200 & \text{otherwise.} \end{cases}$$

5 Symmetric Subset Predicate Encryption with XEBFF

Construction. We present a detailed construction of SSPE with XEBFF called XBPE. This construction is considered an extension of XPE [18]. Let $SE = (SE.Enc, SE.Dec)$ be the PCPA secure symmetric encryption scheme defined in Section 2.2 of [18], a secure PRF $F : \{0, 1\}^\lambda \times \{0, 1\}^k \rightarrow \{0, 1\}^\lambda$ and $\mathcal{M} = \text{'True'}$. The construction contains the following algorithms:

XBPE.Setup(1^λ): On input the security parameter λ , it selects $b, m \in \mathbb{N}$ and runs $XEBFF.setup(b, m)$ to generate the filter. The algorithm outputs master key $msk = (sk, B, h_f, \mathcal{H})$, message space $\mathcal{M} = \text{'True'}$ and $\mathcal{U} = \{0, 1\}^k$.

XBPE.KeyGen(msk, X): On input the master key msk and a predicate set $X \subseteq \mathcal{U}$, let $K \xleftarrow{\$} \{0, 1\}^\lambda$, it generates $sk_X = (sk_X^1, sk_X^2, S)$ corresponding to X where $sk_X^1 = (\bigoplus_{x \in X} F(sk, x)) \oplus K$, $sk_X^2 = SE.Enc(K, 0^\lambda)$, $S = \{S_{x_i} = \{h_0(x_i), h_1(x_i), h_2(x_i)\} \mid x_i \in X, 1 \leq i \leq |X|\}$.

XBPE.Enc(msk, msg, Y): On input the attribute set $Y \subseteq \mathcal{U}$, it computes $Y' = \{y'_i = F(sk, y_i) \mid y_i \in Y, 1 \leq i \leq |Y|\}$ and then insert Y' into empty array B to output the ciphertext $ct \leftarrow XEBFF.construct(B, Y', \mathcal{H}, h_f)$.

XBPE.Dec(sk_X, ct): Given the ciphertext ct and decryption key sk_X . It computes $K' = (\bigoplus_{s \in S} B_Y[s]) \oplus sk_X^1$, $\psi = SE.Dec(K', sk_X^2)$, $msg = \begin{cases} \text{'true'} & \text{if } \psi = 0^\lambda \\ \perp & \text{otherwise.} \end{cases}$

Correctness. The encrypted message can be correctly decrypted iff $X \subseteq Y$. Let $sk_X = (sk_X^1, sk_X^2, S)$ of $X \subseteq \mathcal{U}$ and a ciphertext $ct = B_Y$ of a set $Y \subseteq \mathcal{U}$

$$sk_X^1 = (\bigoplus_{x \in X} F(sk, x)) \oplus K = (\bigoplus_{x \in X, i \in [0, 2]} B_Y[h_i(x)]) \oplus K = (\bigoplus_{s \in S} B_Y[s]) \oplus K$$

Hence, $K' = (\bigoplus_{s \in S} B_Y[s]) \oplus sk_X^1 = K$. Then $\psi = SE.Dec(K', sk_X^2)$ is recovered.

Security Analysis. Let q be a sequence of predicate sets $\mathcal{X} = (X_1, \dots, X_q)$, leakage function of XBPE is $\mathcal{L}_{XBPE} = (|Y|, (\alpha(\mathcal{X}), \beta(X_i, Y), \perp)$ where $|Y|$ is the size of attribute set Y , $\alpha(\mathcal{X})$ is equality pattern of elements in X , indicates which elements among different sets are equal. For example, if $\mathcal{X} = \{\{a, b\}, \{b, c, d\}, \{a, c, f\}\}$ then $\alpha(\mathcal{X}) = \{\{1, 2\}, \{2, 3, 4\}, \{1, 3, 5\}\}$. $\beta(X_i, Y)$ is the result pattern. $\beta(X_i, Y) = 1$ if $X_i \subseteq Y$, and 0 otherwise. The proof of Theorem 1 is given in Appendix A.2.

Theorem 1. Let $\mathcal{L}_{XBPE}$ be the defined leakage function, our XPBE construction is $\mathcal{L}_{XBPE}$ -selectively secure assuming that F is a secure PRF, SE is PCPA secure symmetric encryption scheme and all hash functions are random oracle.

Algorithm 3 EDB Construction

```

FreqTableSetup( $DB$ )
1:  $FreqTable, CMS \leftarrow CMS.Setup(\alpha, \beta)$ 
2: for  $w \in W$  do
3:   for  $id \in DB(w)$  do
4:      $CMS.Add(w)$ 
5:   end for
6: end for
7: return  $FreqTable$ 

EDBSetup( $DB$ )
1: Select  $(K_S, K_I)$  for PRF  $F$  and
2: Parse  $DB$  as  $(id_i, W_i)_{i=1}^d$ .
3:  $T \leftarrow \emptyset$ , where  $T$  is an array indexed by key-
   words from  $W = \bigcup_{i=1}^d W_i$ .
4:  $msk \leftarrow XBPE.setup(1^\lambda)$ 
5:  $XSet \leftarrow \emptyset$ 
6: for all  $w \in W$  do
7:   Initialize  $t \leftarrow \{\}$ 
8:    $K_e \leftarrow F(K_S, w)$ .
9:   Initialize a counter  $c \leftarrow 0$ .
10:  for all  $id \in DB(w)$  in random order do
11:     $e = SE.Enc(K_e, id)$ .
12:     $t \leftarrow t \cup \{e\}$ .
13:    for all  $w' \in W \setminus \{w\}$  and  $id \in DB(w')$ 
14:      do
15:         $xtag \leftarrow F(K_I, w || w' || c)$ 
16:         $XSet \leftarrow XSet \cup \{xtag\}$ .
17:      end for
18:     $c++$ 
19:  end for
20:   $T[w] \leftarrow t$ .
21: end for
22:  $(TSet, K_T) = TSetSetup(1^\lambda, T)$ .
23:  $ct \leftarrow XBPE.Enc(msk, True, XSet)$ 
24: return  $\{K = (K_S, K_I, K_T, msk), EDB = (TSet, ct)\}$ 

```

6 The Construction

6.1 Construction

Our proposed construction is built upon the generic construction of Doris [18] with the construction of frequency table and the s-term acquisition protocol. The construction consists of algorithms as shown in Algorithms 3 and 4.

FreqTableSetup(DB): This algorithm takes the plaintext database and constructs a frequency table using $CMS.Setup$ and $CMS.Add$.

EDBSetup(DB): Given the security parameter 1^λ , the algorithm generates two pseudorandom function (PRF) keys, $K_S, K_E \leftarrow \{0, 1\}^\lambda$ for the PRF function F , together with a master key msk for the XBPE scheme. For each distinct keyword in the database, the client encrypts every associated document id and stores them in an array T , from which TSet is generated. TSet is an expanded inverted index data structure which consists of a list of fixed-length tuples associated with each keyword in a database [5]. With TSet, clients can issue corresponding tokens to retrieve the list related to the queried keyword. The details about TSet is given in Appendix A.1. The client then constructs $xtag$ values by pairing each keyword with a counter in every document, subsequently adding them to XSet. After this, the client encrypts $msg = \text{True}$ with XSet using $XBPE.Enc$ to generate the ciphertext ct . Finally, the algorithm outputs the secret keys K_S, K_I, K_T, msk along with the encrypted database EDB , which consists of TSet and ct .

Search($K, \Phi = (w_1 \wedge \dots \wedge w_n), EDB$): The client obtains the s-term by checking the estimated frequency values in $FreqTable$. Then, the client computes and sends $stag$ and $xtoken$ to the server, where $xtoken$ is generated using $XBPE.KeyGen$. On the server side, the encrypted document identifiers corresponding to the s-term are retrieved. The server then uses $xtoken$ as the decryption key to recover $msg = \text{True}$. If $QSet_c$ is presented in ct , the server verifies that the retrieved

Algorithm 4 Search

```

Search( $K, \Phi, FreqTable$ )
1: Client:
2: Client has  $(K_S, K_I, K_T)$  and query  $\Phi = (w_1 \wedge \dots \wedge w_n)$ .
3:  $stern \leftarrow w_1$ 
4: Initialize  $minCount \leftarrow \infty$ 
5: for  $w \in \Phi$  do
6:    $currCount \leftarrow CMS.Query(w)$ 
7:   if  $currCount < minCount$  then
8:      $minCount \leftarrow currCount$ 
9:    $stern \leftarrow w$ 
10: end if
11: end for
12: Client sends  $stag \leftarrow TSetGetTag(K_T, stern)$  to the server.
13: for  $c = 1, 2, 3, \dots$  until server sends stop do
14:   for  $i = 2, \dots, n$  do
15:      $qtag_c \leftarrow F(K_I, stern || w_i || c)$ 
16:      $QSet_c \leftarrow QSet_c \cup \{qtag_c\}$ 
17:   end for
18:    $xtoken[c] \leftarrow XBPE.KeyGen(msk, QSet_c)$ 
19:   Send  $xtoken[c]$  to server
20: end for
21: Server:
22: Set  $t \leftarrow TSetRetrieve(TSet, stag)$ .
23: for  $c = 1, \dots, |t|$  do
24:    $msg \leftarrow XBPE.Dec(xtoken[c], ct)$ 
25:   if  $msg = True$  then
26:      $Res \leftarrow Res \cup \{e_c\}$ 
27:   end if
28: end for
29: Send  $Res$  to client
30: Client:
31:  $K_{e1} \leftarrow F(K_S, stern)$ 
32: for  $e \in Res$  do
33:    $v \leftarrow SE.Dec(K_{e1}, e)$ 
34:    $FRes \leftarrow FRes \cup \{v\}$ 
35: end for

```

document identifiers contain all the remaining conjunctive terms $w_2, w_3, \dots, w_n$. Finally, the server sends the set of results Res to the client, which decrypts the relevant document identifiers to obtain the final search results.

6.2 Leakage Function Comparison

In this section, we analyse and compare the leakage profile of our scheme with HXT. We shown that our scheme has successfully remove KPRP and IP leakage while introduce a trivial leakage called XEP. Let Q be a sequence queries written as $\Phi = (s, x_2, \dots, x_q)$, where vector s consist of sterms and vector $x_2, \dots, x_q$ consists of x-terms. For the i -th query, we defines $\Phi[i] = (s[i] \wedge x_2[i] \wedge \dots \wedge x_q[i])$ where $1 \leq i \leq Q$, the leakage profile $\mathcal{L} = (|TSet|, |XSet|, SEP, XEP, SRP, WRP)$ is described as follows:

- $|TSet|$ is the size of TSet that comes from TSet initialization process.
- $|XSet|$ is the size of XSet that comes from XBPE encryption process.
- **SEP** is the equality pattern of s-terms that comes from the repetitions when searching for the same s-term in different queries. SEP indicates which queries have the same s-term. For all $i, i' \in [1, Q]$, $SEP[i] = SEP[i']$ if $s[i] = s[i']$. For example, if $s = (a, b, a, c, b)$ $SEP = (1, 2, 1, 3, 2)$
- **XEP** is the equality pattern of x-terms that comes from the location set S_i during decryption key generation process of XBPE. XEP indicates which queries have the same x-term when sharing same s-term. For all $i, i' \in [1, Q]$ and $j, j' \in [2, q]$, $XEP[i, j] = XEP[i', j']$ if $(s[i] = s[i']) \wedge (x_j[i] = x_{j'}[i'])$. For example, $\Phi[1] = (a \wedge b \wedge c \wedge d)$, $\Phi[2] = (a \wedge c \wedge e \wedge b)$, $\Phi[3] = (f \wedge e \wedge d \wedge g)$, $\Phi[4] = (f \wedge d \wedge g \wedge h)$, then

$$XEP = \begin{vmatrix} 1 & 2 & 3 \\ 2 & 4 & 1 \\ 5 & 6 & 7 \\ 6 & 7 & 8 \end{vmatrix}$$

- **SRP** is the result pattern of s-term, which is the search result from TSet matching s-term. $SRP[i] = T[s[i]], i \in [1, Q]$.
- **WRP** is the whole result pattern, which is the search result for the conjunctive queries. $WRP[i] = DB[s[i]] \cap DB[x_2[i]] \cap \dots \cap DB[x_q[i]], i \in [1, Q]$

Before analyzing how our schemes can remove KPRP and IP leakage, we first recall the definition of KPRP leakage in [5, 12] and IP leakage in [15, 5].

- **KPRP** is the keyword-pair result pattern leaks the intersection result of the s-term with each x-term in a query. $KPRP[i] = DB[s[i]] \cap DB[x_j[i]], i \in [1, Q], j \in [2, q]$
- **IP** is the conditional intersection pattern leaks the document identifiers containing s-term of queries sharing same x-terms. For $i, i' \in [1, Q], j, j' \in [2, q]$

$$IP[i, i'] = \begin{cases} DB[s[i]] \cap DB[s[i']] & \text{if } i \neq i' \wedge x_j[i] = x_{j'}[i'] \\ \perp & \text{otherwise.} \end{cases}$$

In OXT, the XSet consists of the hash values of encrypted keyword-id pairs. During a search, the server computes the hash values of the encrypted s-term id and the encrypted token of each x-term. For every pair, the server checks whether it exists in the XSet, allowing it to infer intermediate results. In contrast, our scheme leverages Symmetric Subset Predicate Encryption, which verifies whether a set of tokens appears in the XSet as a whole, rather than checking each token individually. Therefore, the server learns nothing about the intermediate results. In terms of IP leakage, the problem still comes from the construction of XSet. Since XSet uses the keyword-id pairs, when two queries have same x-term with different s-terms, if the s-terms of two queries appear in same documents, when the server generates the hash value, it can obtain the identical values from two queries, leading to inferring of documents matching two s-terms.

As a result, we have successfully removed the KPRP and IP leakage while introducing a new leakage called XEP. However, as stated in [18], *XEP* overstates the true leakage. Because the locations are generated by hash functions, therefore even different s-terms and x-terms in two separate queries can still map to identical locations due to hash collisions.

6.3 Security Analysis

Theorem 2. *Let $\mathcal{L}$ be the leakage function defined above, our protocol Π is $\mathcal{L}$ -semantically-secure against adaptive attacks, assuming that F is secure PRF, $XBPE$ is selective simulation secure, SE is IND-CPA secure, $TSet$ is $\mathcal{L}_{\mathcal{T}}$ -adaptive secure and computationally correct.*

The proof is similar to that of [18] which is given in Appendix A.3.

7 Performance Analysis

7.1 Theoretical Analysis

Computation. We analyze the computational cost of setup and search. All schemes share the TSet generation cost $O(T_{gen})$, but differ in XSet construction. OXT and HXT store document-keyword pairs, scaling with $n = \sum_{w \in W} |DB(w)|$, and incur a setup cost of $O(n)\mathcal{E}$ due to modular exponentiation. HXT further requires PRF and hash operations, increasing the cost to $O(n)(\mathcal{E} + \mathcal{P} + H)$. Doris and our scheme use unique keyword pairs $N = \sum_{1 \leq i \leq j \leq |W|} |DB(w_i) \cap DB(w_j)|$ and SSPE encryption. Doris incurs $O(N)(\mathcal{P} + H + XF)$ due to XOR Filter, while our scheme has $O(N)(\mathcal{P} + H + XEBFF)$ from XEBFF operations.

For a conjunctive query $\phi = w_1 \wedge \dots \wedge w_q$, all schemes first perform a TSet lookup with cost $O(T_q)$ and scale with $(q-1)n_{w_1}$ due to the OXT framework. OXT and HXT require modular exponentiation, with HXT adding a hashing overhead for SHVE key generation, resulting in search costs of $O(T_q) + O((q-1)n_{w_1})\mathcal{E}$ and $O(T_q) + O((q-1)n_{w_1})(\mathcal{E} + H)$, respectively. Doris and our scheme avoid exponentiation, using PRF and XOR instead, improving efficiency. In contrast, with the new XSet construction, Doris and our scheme avoid exponentiation and instead use PRF and XOR operations, making the searches more efficient with the search cost of $O(T_q) + O((q-1)n_{w_1})(\mathcal{P} + \mathcal{X})$.

Server Storage Cost. All schemes follow the OXT framework, requiring both TSet and XSet structures. OXT stores document-keyword pairs, resulting in a storage cost of $O(n)$. HXT follows a similar approach but incorporates a Bloom filter and SHVE to hide KPRP. The XSet is first processed through a Bloom filter and then encrypted, leading to a storage cost of $O(\nu pn)$ where ν and p are parameters of Bloom Filter. On the other hand, Doris and our scheme construct the XSet based on unique keyword pairs within documents. Doris applies an XOR filter to the XSet before encrypting it with SSPE, resulting in a storage cost of $O(\gamma N)$. In contrast, our scheme utilizes XEBFF before encryption, achieving a storage cost of $O(\omega N)$, where ω and γ are configurable parameters of XOR filter and XEBFF, respectively, with $\omega \leq \gamma$. In detail, when encrypting small XSet, the storage overhead of both schemes is $1.23N$ ($\omega = \gamma = 1.23$). For larger XSet (i.e., $N \geq 10^7$), our scheme reduced the storage cost to $1.125N$ ($\omega = 1.125$) due to XEBFF, producing more efficient storage compared to Doris.

SHVE vs XBPE. SHVE [15] and XBPE are predicate-only encryption schemes used for subset membership testing to hide KPRP. Since SHVE employs Bloom Filter to encrypt XSet so that it incurs more storage overhead than our XBPE. In detail, a Bloom Filter of size m with k hash functions stores N elements, with a false positive probability $p \approx (1 - e^{-kN/m})^k$. Given the desired false positive rate p and optimal number of hash functions $k = \frac{m}{N} \ln 2$, the size of Bloom Filter is $m = \frac{N \log_2 p}{\ln 2}$, meaning that under optimal conditions, the Bloom filter requires $1.44N(-\log_2 p)$ bits, or 44% more storage than the theoretical lower bound of $-\log_2 p$ bits per key [2]. In contrast, XEBFF's storage is independent of false positive rate, which instead depends on the fingerprint hash size. As given above, the encrypted XSet storage cost is about $1.23N$ when $N < 10^7$

and it reduces to $1.125N$ when $N \geq 10^7$. Thus, for a given false positive rate, our XBPE is more storage-efficient than SHVE.

7.2 Empirical Analysis



Fig. 1: EDB setup

We run experiments to compare our scheme with Doris [18]. Theoretical comparisons with OXT [5], HXT [15], and Doris [18] are also given in the full version.

Implementation Settings. We build on the Java implementation of OXT [14] to implement our scheme, we also reimplement Doris in Java to ensure a fair comparison. For filtering, we use XOR filter and BFF implementations from FastFilter [10] with an 8-bit fingerprint hash function. We use Count-Min Sketch implementation from Steamlib [8] to build a frequency table with an error rate of 1% and a confidence accuracy rate of 99%. All experiments run on the Enron Email Dataset [4] using an Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz (12 cores) and 16GB of RAM.

Experiment Settings. We compare EDB setup time and search efficiency for two-conjunction and multi-keyword queries. Additionally, we analyze SSPE construction time in Doris (XPE) compared to our proposed XBPE and measure storage costs using the Java Agent Memory Management (JAMM) tool.

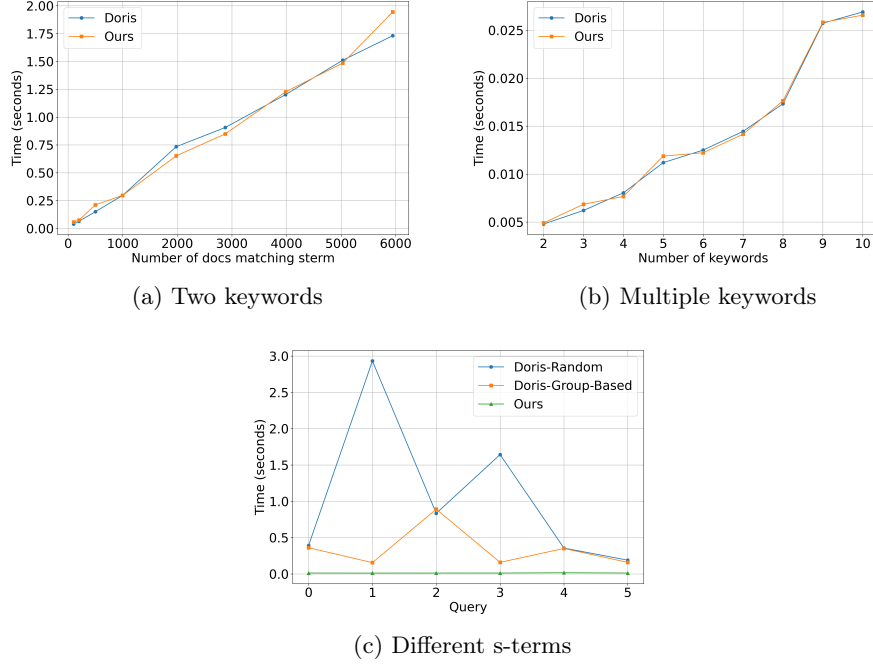


Fig. 2: Search Time

EDB Setup. We test EDB setup time on databases ranging from 1000KB to 8000KB. According to Fig. 1a, the setup time differences between our scheme and Doris are minimal since setup time is dominated by XSet construction rather than SSPE encryption. $Time(EDBSetup) = Time(XSet.Construct) + Time(SSPE.Enc)$. Hence, to highlight the performance differences, we compare the construction time of XPE and XBPE as shown in Fig. 1b. Our results show that XBPE construction is approximately $1.5\times$ faster than XPE. Additionally, since Doris outperforms OXT and HXT by eliminating costly exponentiations [18], we expect our scheme also outperforms OXT and HXT in setup efficiency.

Storage Cost. As shown in Fig. 1c, our scheme significantly reduces storage costs as data size grows. For datasets up to 100KB, storage cost is similar to Doris, but for datasets larger than 100KB, our scheme reduces costs by approximately 8%. On the client side, we use a fixed-size data structure for the Frequency Table, maintaining a consistent 0.107MB memory across all data sizes (Fig. 1d), making it ideal for storage-constrained environments.

Search Performance. For the search evaluation, we use a smaller dataset (250KB) similar to Doris implementation. First, we measure the search performance of two conjunctive keywords while varying the number of documents matching the s-term. We then extend our experiments to multi-keyword conjunctive queries. As shown in Fig. 2a and 2b, our search performance closely

matches that of Doris when they use the same keyword as s-term. Since the primary difference between XEBFF and the XOR filter lies in their construction and storage costs, there is no impact on query time. Next, we evaluate the impact of optimal s-term selection on search efficiency. To test stability, we run multiple searches on a fixed keyword list, shuffling query order each time. We compare our scheme against Doris with random s-term selection (defaulting to the first keyword) and group-based selection (as discussed in OXT), which uses five Bloom filters for keywords with frequencies below 500, 1000, 2000, 4000, and 8000. Although we can improve the accuracy of the group-based selection method by increasing the number of filters, this comes at the cost of additional performance overhead, and no standardized approach exists for balancing cost and accuracy. In contrast, our CMS approach provides a more systematic and effective solution in identifying the least frequent keyword. As a result, our scheme maintains stable search performance as shown in Fig. 2c.

8 Conclusion

In conclusion, we formalize the construction of XOR Filter and Binary Fuse Filter into a novel structure, XEBFF, and integrate it into Symmetric Subset Predicate Encryption (SSPE) to optimize storage efficiency. We further present a non-interactive conjunctive symmetric searchable encryption scheme that eliminates both KPRP and IP leakage, while leveraging the compactness of SSPE. To enhance performance, we incorporate a Count-Min Sketch-based method for selecting the least frequent keywords, achieving sublinear search complexity. Experimental results show that our scheme significantly outperforms OXT and Doris in both construction time and storage overhead.

Several directions remain for future investigation. Firstly, evaluating the scheme on extremely large datasets would help stress-test its scalability, revealing potential bottlenecks in construction time and offering clearer insights into performance at scale. Secondly, broader empirical evaluations are needed to guide practical parameter tuning—such as optimal filter sizes, hash counts, and storage trade-offs—which are critical for real-world deployment. These evaluations can inform the development of adaptive or data-aware tuning strategies, ultimately enhancing the scheme’s practicality and robustness.

References

1. Ballard, L., Kamara, S., Monrose, F.: Achieving efficient conjunctive keyword searches over encrypted data. In: Qing, S., Mao, W., López, J., Wang, G. (eds.) *Information and Communications Security*, 7th International Conference, ICICS 2005, Beijing, China, December 10-13, 2005, *Proceedings. Lecture Notes in Computer Science*, vol. 3783, pp. 414–426. Springer (2005). https://doi.org/10.1007/11602897_35
2. Broder, A., Mitzenmacher, M.: Network Applications of Bloom Filters: A Survey. *Internet Mathematics* **1**(4), 485–509 (2003). <https://doi.org/10.1080/15427951.2004.10129096>

3. Byun, J.W., Lee, D.H., Lim, J.: Efficient Conjunctive Keyword Search on Encrypted Data Storage System. In: Atzeni, A.S., Lioy, A. (eds.) *Public Key Infrastructure, Third European PKI Workshop: Theory and Practice, EuroPKI 2006*, Turin, Italy, June 19-20, 2006, Proceedings. *Lecture Notes in Computer Science*, vol. 4043, pp. 184–196. Springer (2006). https://doi.org/10.1007/11774716_15
4. CALO Project: Enron Email Dataset. <https://www.cs.cmu.edu/~enron/> [online] (2015)
5. Cash, D., et al.: Highly-scalable searchable symmetric encryption with support for boolean queries. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference*, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 8042, pp. 353–373. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_20
6. Cormode, G., Muthukrishnan, S.: An Improved Data Stream Summary: The Count-Min Sketch and its Applications. *Journal of Algorithms* **55**(1), 58–75 (2005). <https://doi.org/10.1016/j.jalgor.2003.12.001>
7. Golle, P., Staddon, J., Waters, B.: Secure conjunctive keyword search over encrypted data. In: Jakobsson, M., Yung, M., Zhou, J. (eds.) *Applied Cryptography and Network Security, Second International Conference, ACNS 2004*, Yellow Mountain, China, June 8-11, 2004, Proceedings. *Lecture Notes in Computer Science*, vol. 3089, pp. 31–45. Springer (2004). https://doi.org/10.1007/978-3-540-24852-1_3
8. Google: Stream Library. <https://github.com/addthis/stream-lib> [online] (2017)
9. Graf, T.M., Lemire, D.: Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters. *Journal of Experimental Algorithmics* **25**(1), 1–16 (2020). <https://doi.org/10.1145/3376122>
10. Graf, T.M., Lemire, D.: Fast Approximate Membership Filters in Java. https://github.com/FastFilter/fastfilter_java [online] (2021)
11. Graf, T.M., Lemire, D.: Binary Fuse Filters: Fast and Smaller Than Xor Filters. *Journal of Experimental Algorithmics* **27**(1), 1–15 (2022). <https://doi.org/10.1145/3510449>
12. Kamara, S., Moataz, T.: Boolean searchable symmetric encryption with worst-case sub-linear complexity. In: Coron, J., Nielsen, J.B. (eds.) *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Paris, France, April 30 - May 4, 2017, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 10212, pp. 94–124 (2017). https://doi.org/10.1007/978-3-319-56617-7_4
13. Katz, J., Maffei, M., Malavolta, G., Schröder, D.: Subset predicate encryption and its applications. In: Capkun, S., Chow, S.S.M. (eds.) *Cryptology and Network Security - 16th International Conference, CANS 2017*, Hong Kong, China, November 30 - December 2, 2017, Revised Selected Papers. *Lecture Notes in Computer Science*, vol. 11261, pp. 115–134. Springer (2017). https://doi.org/10.1007/978-3-030-02641-7_6
14. Lai, S., et al.: Boolean Query SSE Schemes. <https://github.com/MonashCybersecurityLab/Boolean-Query-SSE> [online] (2018)
15. Lai, S., et al.: Result pattern hiding searchable encryption for conjunctive queries. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018*, Toronto, ON, Canada, October 15-19, 2018. pp. 745–762. ACM (2018). <https://doi.org/10.1145/3243734.3243753>

16. Ma, C., Gu, Y., Li, H.: Practical Searchable Symmetric Encryption Supporting Conjunctive Queries Without Keyword Pair Result Pattern Leakage. *IEEE Access* **8**, 107510–107526 (2020). <https://doi.org/10.1109/access.2020.3001014>
17. Wang, P., Wang, H., Pieprzyk, J.: Keyword field-free conjunctive keyword searches on encrypted data and extension for dynamic groups. In: Franklin, M.K., Hui, L.C.K., Wong, D.S. (eds.) *Cryptology and Network Security*, 7th International Conference, CANS 2008, Hong-Kong, China, December 2-4, 2008. *Proceedings. Lecture Notes in Computer Science*, vol. 5339, pp. 178–195. Springer (2008). https://doi.org/10.1007/978-3-540-89641-8_13
18. Wang, Y., Sun, S.F., Wang, J., Chen, X., Liu, J.K., Gu, D.: Practical non-interactive encrypted conjunctive search with leakage suppression. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024*, Salt Lake City, UT, USA, October 14-18, 2024. pp. 4658–4672. ACM (2024). <https://doi.org/10.1145/3658644.3670355>
19. Zhang, Y., Katz, J., Papamanthou, C.: All your queries are belong to us: The power of file-injection attacks on searchable encryption. In: Holz, T., Savage, S. (eds.) *25th USENIX Security Symposium, USENIX Security 16*, Austin, TX, USA, August 10-12, 2016. pp. 707–720. USENIX Association (2016)

A Appendix

A.1 TSet

According to [5], TSet consists of three algorithms. $TSetSetup(1^\lambda, T)$ takes an array $\mathbf{T} = \{w, (id_1, id_2, \dots, id_{|T_w|})\}_{w \in W}$ and places id_i of $\mathbf{T}[w]$ into bucket $TSet[\theta]$ where θ is computed from two PRFs, $F, \bar{F}$ and one hash function H . $TSetGetTag(K_T, w)$ takes key K_T and queried keyword w as input and outputs a search token $stag = \bar{F}(K_T, w)$. $TSetRetrieve(TSet, stag)$ takes $TSet$ and search token $stag$ corresponding to w as inputs. Then the tuple $\mathbf{T}[w]$ associated with w are returned.

Correctness. Similar to SSE, TSet is computationally correct if for all PPT adversaries $\mathcal{A}$ can win the game $\mathbf{Cor}_{\mathcal{A}}^{TSet}$ with negligible probability $\Pr[\mathbf{Cor}_{\mathcal{A}}^{TSet} = 1] \leq \text{negl}(\lambda)$. In this game, adversary $\mathcal{A}$ selects its own array $\mathbf{T}$ and obtains $TSet$ using $TSetSetup$. Then the adversary adaptively chooses queried keywords w and runs $TSetGetTag$ and $TSetRetrieve$ to get the result Res . If Res and $\mathbf{T}[w]$ are not identical, the game outputs 1. Otherwise, it outputs 0.

Security. Cash et al. [5] define the leakage of TSet instantiation is the number of all pairs of keyword and identifier in the database, $\mathcal{L}_T = \{\mathcal{L}_T(\mathbf{T}), \mathcal{L}_T(\mathbf{T}, q)\}$ where $\mathcal{L}_T(\mathbf{T}) = \sum_{w \in W} |DB(w)|$ and $\mathcal{L}_T(\mathbf{T}, q)$ is leakage from queries q .

Theorem 3. *According to Theorem 7 of [5], for any keyword sequences, including an empty sequence, the TSet instantiation is $\mathcal{L}_T$ -adaptively-secure assuming that F and $\bar{F}$ are secure PRFs and H is a random oracle.*

A.2 Proof of theorem 2

Game G_0 . This game is identical to the real game. A list of tables $\mathcal{T}$ corresponding to the array segment is built to simulate the behaviors of hash functions.

$$\Pr[G_0 = 1] = \Pr[\mathbf{Real}_{\mathcal{A}}^{XBPE}(\lambda) = 1]$$

Game G_1 . This game modifies the way of generating $sk_{X_i}^1$. In detail, if $X_i \subseteq Y$, $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $sk_{X_i}^1 = (\bigoplus_{s \in S} B[s]) \oplus K$. Otherwise, $sk_{X_i}^1 \xleftarrow{\$} \{0, 1\}^\lambda$. Since F is a secure PRF function and h_f is random oracle, we have

$$|\Pr[G_1 = 1] - \Pr[G_0 = 0]| \leq \text{negl}(\lambda)$$

Game G_2 . This game modifies the way of generating $sk_{X_i}^2$. In detail, if $X_i \subseteq Y$ then $sk_{X_i}^2 = SE.Enc(K, 0^\lambda)$ where $K \xleftarrow{\$} \{0, 1\}^\lambda$. Otherwise, $sk_{X_i}^2 \xleftarrow{\$} \{0, 1\}^\lambda$. Since SE is PCPA secure, we have

$$\Pr[G_2 = 1] - \Pr[G_1 = 0] \leq \text{negl}(\lambda)$$

Game G_3 . This game modifies the way of generating multiset S . For the decryption key query X_i , an integer is assigned to each element of X_i according to the order in which it appears. Therefore, we can construct the equality patterns of queries, then we can use it to produce S .

$$\Pr[G_2 = 1] = \Pr[G_3 = 0]$$

Game G_4 . This game modifies the way of generating ciphertext. Instead of inserting $Y' = \{F(sk, y_i)\}$, we insert $Y' \xleftarrow{\$} \{0, 1\}^\lambda$. Due to security of PRF,

$$\Pr[G_3 = 1] - \Pr[G_4 = 0] \leq \text{negl}(\lambda)$$

Simulator. The ideal game with simulator $\mathcal{S}_X$ is defined as follows. Firstly, $\mathcal{A}$ selects Y and gives it to simulator $\mathcal{S}_X$. Then, $\mathcal{S}_X.Setup(|Y|)$ initializes an array B of size b and a list of m tables $\mathcal{T} = \{T_0, \dots, T_m\}$. As mentioned in section 3, the

optimal values of b and m depend on $|Y|$: $b \approx \begin{cases} 1.23 \cdot |S| + 32 & \text{if } |Y| < 10^7 \\ 1.125 \cdot |S| & \text{otherwise.} \end{cases}$, $m =$

$\begin{cases} 3 & \text{if } |Y| < 10^7 \\ 200 & \text{otherwise.} \end{cases}$. The simulator runs $\mathcal{S}_X.Enc(|Y|)$ to output B as ciphertext

$B[i] \xleftarrow{\$} \{0, 1\}^\lambda$. Finally, the simulator executes $\mathcal{S}_X.KeyGen(\alpha(\mathcal{X}), \beta(X_i, Y))$ to generate $sk_{X_i} = (sk_{X_i}^1, sk_{X_i}^2, S)$. To compute S , for each $j \in \alpha(X_i)$, it looks up the tables $\mathcal{T}$. If j exists in three consecutive tables, $T_k[j]$, $T_{k+1}[j]$, $T_{k+2}[j]$ are inserted to location set S . Otherwise $k \xleftarrow{\$} [0, m]$ and $T_k[j] \xleftarrow{\$} [0, \frac{(k+1) \cdot b}{m})$, $T_{k+1}[j] \xleftarrow{\$} [\frac{(k+1) \cdot b}{m}, \frac{(k+2) \cdot b}{m})$, $T_{k+2}[j] \xleftarrow{\$} [\frac{(k+2) \cdot b}{m}, \frac{(k+3) \cdot b}{m})$. If $\beta(X_i, Y) = 1$ the simulator sets $sk_{X_i}^1 = (\bigoplus_{s \in S} B[s]) \oplus K$, $sk_{X_i}^2 = SE.Enc(K, 0^\lambda)$ where $K \xleftarrow{\$} \{0, 1\}^\lambda$, otherwise $sk_{X_i}^1 \xleftarrow{\$} \{0, 1\}^\lambda$ and $sk_{X_i}^2 \xleftarrow{\$} \{0, 1\}^\lambda$. At this point, the simulator has the same distribution as G_4 . By combining all the above games, the proof of Theorem is concluded.

A.3 Proof of theorem 3

Game G_0 . This game is designed to be identical to the real one. Specifically, the game constructs TSet and XSet similar to the real protocol. The game generates transcript similar to the search protocol except that the final search $FRes[i]$ is computed by looking up the value of $DB(s[i]) \cap \bigcap_{j=2}^q DB(x_j[i])$ instead of decrypting encrypted results. This game also build a minor bookkeeping changes which records the permutations σ in a vector $WPerms$ indexed by keywords. Hence, assuming no false positives occur. We have

$$\Pr[G_0 = 1] \leq \Pr[\mathbf{Real}_{\mathcal{A}}^I(\lambda) = 1] + \text{negl}(\lambda).$$

Game G_1 . This game replaces PRFs $F(K_I, \cdot)$ and $F(K_S, \cdot)$ with random functions. Due to secure PRFs, we have

$$|\Pr[G_1 = 1] - \Pr[G_0 = 1]| \leq \text{negl}(\lambda).$$

Game G_2 . This game replaces the input of $SE.Enc$ with a constant string 0^λ . Since SE is IND-CPA secure, we have

$$|\Pr[G_2 = 1] - \Pr[G_1 = 1]| \leq \text{negl}(\lambda).$$

Game G_3 . This game replaces $TSetSetup$ and $TSetGetTag$ with a simulator $\mathcal{S}_T$ on input of $\mathcal{L}_T = \{\mathcal{L}_T(\mathbf{T}), \mathcal{L}_T(\mathbf{T}, q)\}$ where $\mathcal{L}_T(\mathbf{T}) = |TSet|$ and $\mathcal{L}_T(\mathbf{T}, q) = (SEP, SRP)$. As proven in OXT [5], there exists an efficient simulator such that

$$|\Pr[G_3 = 1] - \Pr[G_2 = 1]| \leq \text{negl}(\lambda)$$

Game G_4 . This game changes the way of generating XSet and xtoken. The adversary $\mathcal{A}$ assigns random elements to $XSet$. For xtoken, if there are some previous queries i' such that $(s[i] = s[i']) \wedge (x_j[i] = x_{j'}[i'])$ then $qtag_{c,j}[i] = qtag_{c,j'}[i']$. The value of $qtag_{c,j}[i]$ must be selected from pre-defined $XSet$ if its $id_{\sigma(c)}$ is the final search result, otherwise $qtag_{c,j}[i]$ is randomly selected.

$$\Pr[G_4 = 1] = \Pr[G_3 = 1]$$

Game G_5 . This game invokes simulator S_X of XBPE to generate ct and $xtoken[c]$. In particular, ct is generated based on the size of XSet. $xtoken[c]$ is generated based on XEP and SRP. According to description of G_4 and G_5 , we have the real and ideal games of $XBPE$ respectively, hence

$$|\Pr[G_5 = 1] - \Pr[G_4 = 1]| \leq \text{negl}(\lambda)$$

Simulator. Simulator uses $\mathcal{L}_T$ to generate TSet and STags. It also generate ct by invoking S_X . To compute $xtoken[c]$, we need to obtain $\alpha(QSet_c)$ and $\beta(QSet_c, XSet)$. $\alpha(QSet_c)$ can be obtained from XEP. For $\beta(QSet_c, XSet)$, we first retrieve $id_{\sigma(c)}$ from bookkeeping changes $WPerms[SEP[i]]$ and check whether $id_{\sigma(c)} \in WRP[i] \cap SRP[i]$. As a result, the simulator has same distribution as G_5 , we can conclude the proof.